

Aashu Yadav
T050

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Full Stack development MERN Practical

M2 Practical-9

Roll No.:T050	Name: Aashu Yadav
Class:TYIT	Batch:1
Date of Assignment: 30-9-2026	Date/Time of Submission: 30-9-2026

9.User Data Management

- Create an API to add user details using Express and MongoDB.
- Design a React form to send user data to the backend.
- Retrieve and display user data from MongoDB.
- Implement an update operation for existing records.
- Implement the delete operation for selected records.
- Display updated records dynamically in React.

Code:-

```
use UserDB
db.users.deleteMany({})
db.users.insertMany([
  {
    name: "Aashu",
    email: "aashu@gmail.com",
    city: "Mumbai"
  },
  {
    name: "Riya",
    email: "riya@gmail.com",
    city: "Pune"
  },
  {
    name: "Sneha",
    email: "sneha@gmail.com",
    city: "Nashik"
  }
])
```

```
db.users.insertMany([
  {
    name: "Aashu",
    email: "aashu@gmail.com",
    city: "Mumbai"
  },
  {
    name: "Riya",
    email: "riya@gmail.com",
    city: "Pune"
  },
  {
    name: "Sneha",
    email: "sneha@gmail.com",
    city: "Nashik"
  }
])
```

Step 2 - Create backend

- Create a folder practical9 & open that in vs code
 - Press ctrl+~ to open terminal & follow next step
 - Run -> npm init -y to create package.json file.
 - Add server.js file
 - Run command -> npm install mongoose express cors
 - Click + icon(new file) near practical9 & name it server.js
 - Add following code into server.js file
- ```
const express = require('express');
Code
server.js
const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
const app = express();
```

```

const PORT = 3000;
// Middleware
app.use(cors());
app.use(express.json());
// MongoDB Connection
mongoose.connect('mongodb://127.0.0.1:27017/User
DB');
.then(() => {
console.log('MongoDB Connected
Successfully');
})
.catch((error) => {
console.log('MongoDB Connection Failed');
console.log(error);
});
// GET - Retrieve Users
app.get('/users', async (req, res) => {
try {
const users = await mongoose.connection.db
.collection('users')
.find()
.toArray();
res.json(users);
}
catch (error) {
res.status(500).json({message: 'Error fetching
users'});
}
});
// POST - Add User
app.post('/users', async (req, res) => {
try {
const user = req.body;
await mongoose.connection.db
.collection('users')
.insertOne(user);
res.json({message: 'User added
successfully'});
}
catch (error) {
res.status(500).json({message: 'Error adding
user'});
}
});
// PUT - Update User
app.put('/users/:id', async (req, res) => {
try {
const id = new mongoose.Types.ObjectId(req.params.id);
const updatedUser = req.body;
await mongoose.connection.db
.collection('users')
.updateOne({ _id: id },
{
$set: {

```

```

name: updatedUser.name,
email: updatedUser.email,
city: updatedUser.city
}
}
);
res.json({message: 'User updated
successfully'});
}

catch (error) {
res.status(500).json({message: 'Error updating
user'});
}
});
// DELETE - Delete User
app.delete('/users/:id', async (req, res) => {
try {
const id = new mongoose.Types.ObjectId(req.params.id);
await mongoose.connection.db
.collection('users')
.deleteOne({
_id: id
});
res.json({message: 'User deleted
successfully'});
}
catch (error) {
res.status(500).json({message: 'Error deleting
user'});
}
});
// Start Server
app.listen(PORT, () => {
console.log('Server is running on
http://localhost:${PORT}');
});

```

### Step 3 - Create frontend

- 1) In terminal create react project type `npm create vite@latest frontend`
- 2) Choose react, javascript, ESLint
- 3) In terminal move to frontend & install axios  
`npm install axios`
- 4) expand src & replace App.jsx code with following

#### Code:

##### App.jsx

```

import { useEffect, useState } from 'react';
import axios from 'axios';

function App() {
// Store all users
const [users, setUsers] = useState([]);
// Form data

```

```

const [name, setName] = useState('');
const [email, setEmail] = useState('');
const [city, setCity] = useState('');
// Store ID of user being edited
const [editId, setEditId] = useState(null);
// Get Users from MongoDB
const getUsers = () => {
 axios.get('http://localhost:3000/users')
 .then((response) => {
 setUsers(response.data);
 })
 .catch((error) => {
 console.log(error);
 });
};

// Load users when page starts
useEffect(() => { getUsers(); }, []);
// Add or Update User
const saveUser = () => {
 const user = {

 name: name,
 email: email,
 city: city
 };
 // UPDATE existing user
 if (editId) {
 axios.put('http://localhost:3000/users/' + editId, user)
 .then((response) => {
 alert(response.data.message);
 clearForm();
 getUsers();
 })
 .catch((error) => { console.log(error); });
 }
 // ADD new user
 else {
 axios.post('http://localhost:3000/users', user)
 .then((response) => {
 alert(response.data.message);
 clearForm();
 getUsers();
 })
 .catch((error) => { console.log(error); });
 }
};

// Edit User
const editUser = (user) => {
 setEditId(user._id);
 setName(user.name);
 setEmail(user.email);
 setCity(user.city);
};

// Delete User

```

```

const deleteUser = (id) => {
 if (window.confirm('Are you sure you want to delete this user?')) {
 axios.delete('http://localhost:3000/users/' + id)
 .then((response) => {
 alert(response.data.message);
 getUsers();
 })
 .catch((error) => { console.log(error); });
 }
};

// Clear Form
const clearForm = () => {
 setName('');
 setEmail('');
 setCity('');
 setEditId(null);
};

return (
 <div>
 <h1>User Management System</h1>
 <div>
 <h2>
 {editId ? 'Update User' : 'Add User'}
 </h2>
 <input type="text"
 placeholder="Enter Name"
 value={name}
 onChange={e => setName(e.target.value)}
 />

 <input type="email"
 placeholder="Enter Email"
 value={email}
 onChange={e => setEmail(e.target.value)}
 />

 <input type="text"
 placeholder="Enter City"
 value={city}
 onChange={e => setCity(e.target.value)}
 />

 <button onClick={saveUser}>
 {editId ? 'Update User' : 'Add User'}
 </button>
 <button
 {editId & & 'Cancel'}
 onClick={clearForm}>
 </button>
 </div>
 </div>
);

```

```

<hr />
{/* Display Users */}
<h2>User List</h2>
<table border="1">
<thead>
<tr>
<th>Name</th>
<th>Email</th>
<th>City</th>
<th>Actions</th>
</tr>
</thead>
<tbody>
{users.map((user) => (
<tr key={user._id}>
<td>{user.name}</td>
<td>{user.email}</td>
<td>{user.city}</td>
<td>
<button onClick={() => editUser(user)}>Edit</button>
<button onClick={() => deleteUser(user._id)}>Delete</button>
</td>
</tr>
))}
</tbody>
</table>
</div>
);
}
export default App;

```

#### Step 4 - Execution part

- 1) Split terminal
- 2) execute backend(server) means node server.js
- 3) execute frontend using npm run dev

Output

## User Management System

### Add User

Enter Name

Enter Email

Enter City

Add User

localhost:5173 says

User added successfully

OK

### User List

| Name  | Email           | City   | Actions                           |
|-------|-----------------|--------|-----------------------------------|
| Aashu | aashu@gmail.com | Mumbai | <div>Edit</div> <div>Delete</div> |
| Riya  | riya@gmail.com  | Pune   | <div>Edit</div> <div>Delete</div> |
| Sneha | sneha@gmail.com | Nashik | <div>Edit</div> <div>Delete</div> |

### User List

| Name  | Email           | City   | Actions                           |
|-------|-----------------|--------|-----------------------------------|
| Aashu | aashu@gmail.com | Mumbai | <div>Edit</div> <div>Delete</div> |
| Riya  | riya@gmail.com  | Pune   | <div>Edit</div> <div>Delete</div> |
| Sneha | sneha@gmail.com | Nashik | <div>Edit</div> <div>Delete</div> |
| Diya  | diya@gmail.com  | Delhi  | <div>Edit</div> <div>Delete</div> |